

Gaussian Elimination and Back Substitution

Numerical Solution of Simultaneous Linear Equations

Cristian Vargas

Universidad de Pamplona
Facultad de Ciencias Básicas
Computational Physics II

September 7, 2026

Outline

- 1 Motivation and Problem Statement
- 2 Mathematical Foundation
- 3 Gaussian Elimination
- 4 Back Substitution
- 5 Python Implementation
- 6 Computational Cost
- 7 Summary

The Problem: Simultaneous Linear Equations

We want to solve a system of n linear equations in n unknowns:

$$\begin{array}{rcl} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n & = & b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n & = & b_2 \\ & \vdots & \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n & = & b_n \end{array} \quad \Longleftrightarrow \quad Ax = b$$

Why not Cramer's rule?

Cramer's rule requires computing $n + 1$ determinants by cofactor expansion, at a cost of $O(n!)$. Gaussian elimination solves the same system at a cost of $O(n^3)$. For $n = 20$, $n!$ is already far larger than the total number of floating point operations a computer can perform in a reasonable time – this is the concrete justification for abandoning the exact-determinant approach in favor of elimination.

Elementary Row Operations

The method rests on three operations that transform the system without changing its solution set:

- ❶ **Row swap:** $F_i \leftrightarrow F_j$
- ❷ **Row scaling:** $F_i \rightarrow \lambda F_i, \quad \lambda \neq 0$
- ❸ **Row combination:** $F_i \rightarrow F_i + \lambda F_j$

Equivalent systems

Two linear systems are *equivalent* if they have exactly the same solution set. Each elementary operation corresponds to left-multiplying the system by an invertible matrix E : $EAx = Eb$. Because E is invertible,

$$Ax = b \iff EAx = Eb.$$

Why Row Combination Preserves the Solution

Consider $F_i \rightarrow F_i + \lambda F_j$. Suppose x satisfies both F_i and F_j simultaneously.

- Then x also satisfies $F_i + \lambda F_j$, since it is a linear combination of two true statements.
- The transformation is **invertible**: applying $F_i \rightarrow F_i - \lambda F_j$ recovers the original row exactly.
- Because the map is invertible, no solution is created or lost – the new system is equivalent to the old one.

Key idea

This is the mathematical guarantee that lets us freely manipulate rows during elimination: every operation used is reversible.

Goal: Upper Triangular Form

We use row operations to transform A into an upper triangular matrix U :

$$U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{pmatrix}$$

Why triangular?

In this form the last equation involves a *single* unknown, the second-to-last involves two (one already known), and so on. This cascading structure is precisely what back substitution exploits.

The Elimination Formula

For pivot column k and row $i > k$, define the multiplier

$$m_{ik} = \frac{a_{ik}}{a_{kk}}, \quad F_i \rightarrow F_i - m_{ik}F_k.$$

Where m_{ik} comes from

We require the new entry in position (i, k) to vanish:

$$a_{ik} - m_{ik} a_{kk} = 0 \implies m_{ik} = \frac{a_{ik}}{a_{kk}}.$$

This is not an arbitrary choice: it is the *unique* value of m_{ik} that achieves the elimination goal.

Requirement: $a_{kk} \neq 0$ (the pivot). If the pivot is zero, we swap with a row below having a nonzero entry in that column (*pivoting*).

Worked Example: Setup

Solve:

$$2x_1 + x_2 - x_3 = 8$$

$$-3x_1 - x_2 + 2x_3 = -11$$

$$-2x_1 + x_2 + 2x_3 = -3$$

Augmented matrix

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array} \right]$$

Worked Example: Elimination Steps

Column 1 (pivot $a_{11} = 2$):

$$m_{21} = \frac{-3}{2} = -1.5, \quad m_{31} = \frac{-2}{2} = -1$$

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 2 & 1 & 5 \end{array} \right]$$

Column 2 (pivot $a_{22} = 0.5$): $m_{32} = \frac{2}{0.5} = 4$

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 0 & -1 & 1 \end{array} \right]$$

The Back Substitution Formula

Once $Ux = c$ is triangular, the last row gives

$$x_n = \frac{c_n}{u_{nn}}.$$

In general, once $x_{i+1}, \dots, x_n$ are known:

$$x_i = \frac{c_i - \sum_{j=i+1}^n u_{ij}x_j}{u_{ii}}$$

Why backward?

Row i needs $x_{i+1}, \dots, x_n$ already known. Only the last row depends on nothing else – it is the unique valid starting point. This is a logical consequence of the triangular structure, not a convenience choice.

Worked Example: Back Substitution

Triangular system:

$$2x_1 + x_2 - x_3 = 8, \quad 0.5x_2 + 0.5x_3 = 1, \quad -x_3 = 1$$

Step by step

$$x_3 = -1$$

$$0.5x_2 + 0.5(-1) = 1 \Rightarrow x_2 = 3$$

$$2x_1 + 3 - (-1) = 8 \Rightarrow x_1 = 2$$

Check: $2(2) + 3 - (-1) = 8$, $-3(2) - 3 + 2(-1) = -11$, $-2(2) + 3 + 2(-1) = -3$. ✓ All three original equations hold.

Forward Elimination: Setup

```
1  # We work on copies so the original A and b are never modified.
2  U = np.array(A, dtype=float).copy()
3  c = np.array(b, dtype=float).copy()
4  n = len(c)
5
6  if verbose:
7      print("Initial augmented matrix [A | b]:")
8      print(np.column_stack((U, c)))
9      print()
10
11  # Loop over pivot columns k = 0, 1, ..., n-2
12  # (in 0-indexed Python; this corresponds to k = 1, ..., n-1 in the notes)
13  for k in range(n - 1):
```

Forward Elimination: Partial Pivoting

```
1  # ---- Partial pivoting ----
2  # We look for the row with the LARGEST absolute value in column k,
3  # among rows k, k+1, ..., n-1. This is not required by the pure
4  # mathematics (any nonzero pivot works), but it drastically reduces
5  # floating point rounding error, because dividing by a very small
6  # pivot amplifies numerical noise.
7
8  pivot_row = np.argmax(np.abs(U[k:, k])) + k
9  if np.isclose(U[pivot_row, k], 0.0):
10     raise ValueError("Matrix is singular: no nonzero pivot available "
11                      f"in column {k}.")
12
13  if pivot_row != k:
14     U[[k, pivot_row]] = U[[pivot_row, k]]
15     c[[k, pivot_row]] = c[[pivot_row, k]]
16     if verbose:
17         print(f"Swap row {k} <-> row {pivot_row} (partial pivoting)")
```

Forward Elimination: Elimination Below the Pivot

```
1      # ---- Elimination below the pivot ----
2      for i in range(k + 1, n):
3          # m_ik is the unique factor that makes the new (i, k) entry zero:
4          #       $U[i, k] - m_{ik} * U[k, k] = 0 \Rightarrow m_{ik} = U[i, k] / U[k, k]$ 
5          m_ik = U[i, k] / U[k, k]
6
7          U[i, k:] = U[i, k:] - m_ik * U[k, k:]
8          c[i] = c[i] - m_ik * c[k]
9
10         if verbose:
11             print(f"R{i} <- R{i} - ({m_ik:.4f}) * R{k}")
12
13     if verbose:
14         print("Augmented matrix after this stage:")
15         print(np.column_stack((U, c)))
16         print()
17
18     return U, c
```

Back Substitution (Python)

```
1  U = np.array(U, dtype=float)
2  c = np.array(c, dtype=float)
3  n = len(c)
4  x = np.zeros(n)
5
6  # We must start at the LAST row, because it is the only equation
7  # that involves a single unknown ( $x_{n-1}$  in 0-indexed notation).
8  # Every other row depends on unknowns that come after it, so it can
9  # only be solved once those later unknowns are already known.
10 for i in range(n - 1, -1, -1):
11     #  $\sum_{j=i+1}^{n-1} U[i, j] * x[j]$  -- the "already known" part
12     known_sum = np.dot(U[i, i + 1:], x[i + 1:])
13     x[i] = (c[i] - known_sum) / U[i, i]
14
15     if verbose:
16         print(f"x[{i}] = (c[{i}] - sum) / U[{i},{i}] "
17               f"= ({c[i]:.4f} - {known_sum:.4f}) / {U[i, i]:.4f} "
18               f"= {x[i]:.4f}")
19
20 return x
```

Running the Example

```
1 import numpy as np
2 from gaussian_elimination import gaussian_elimination
3 from back_substitution import back_substitution
4
5
6 def solve(A, b, verbose=True):
7     """Solve A x = b using Gaussian elimination + back substitution."""
8     U, c = gaussian_elimination(A, b, verbose=verbose)
9     x = back_substitution(U, c, verbose=verbose)
10    return x
```

Key idea

Output: Final solution: $\begin{bmatrix} 2. & 3. & -1. \end{bmatrix}$, with residual $\|Ax - b\|_{\infty} \approx 10^{-16}$ (floating point round-off, not a computational error).

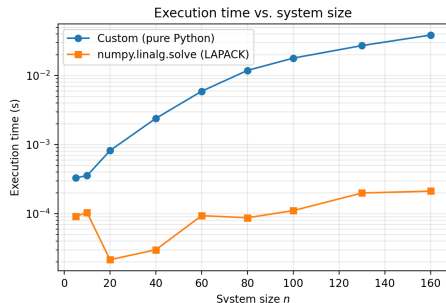
Operation Count

- Forward elimination: $\frac{2n^3}{3} + O(n^2)$ arithmetic operations.
- Back substitution: $O(n^2)$ operations – negligible compared to elimination for large n .

Key idea

Because elimination dominates the cost, solving many systems $Ax = b_1, Ax = b_2, \dots$ with the *same* A motivates saving the elimination work once – this is the idea behind the *LU* decomposition, a direct extension of this method.

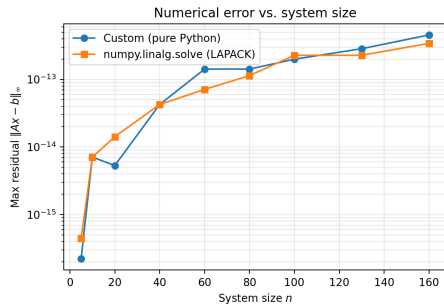
Benchmark: Execution Time vs. System Size



Interpretation

The pure-Python teaching implementation follows the expected $O(n^3)$ growth but with a much larger constant than `numpy.linalg.solve` (compiled LAPACK routines). The *algorithm* is identical; only the *implementation* efficiency differs, by orders of magnitude.

Benchmark: Numerical Error vs. System Size



Interpretation

Both methods show residuals near machine precision, growing slowly with n as round-off accumulates over more operations. Accuracy is comparable because both apply partial pivoting on well-conditioned (diagonally dominant) systems.

Summary

- 1 Elementary row operations preserve the solution set because they correspond to invertible transformations.
- 2 Gaussian elimination reduces $Ax = b$ to triangular form $Ux = c$ using multipliers $m_{ik} = a_{ik}/a_{kk}$ derived directly from the elimination goal.
- 3 Back substitution solves the triangular system starting from the last equation, the only one with a single unknown.
- 4 Cost: $O(n^3)$ for elimination, $O(n^2)$ for substitution – vastly cheaper than Cramer's rule's $O(n!)$.
- 5 Implementation quality (vectorization, compiled libraries) affects speed by orders of magnitude without changing the underlying mathematics or, on well-conditioned systems, the accuracy.

Thank you

Questions?